Project Summary: HerculesMotoCorp Database Design and Implementation

Client Overview

HerculesMotoCorp is a leading U.S.-based automobile retailer and garage services provider. Their operations include custom vehicle manufacturing and sales. Given the scale and legal requirements of their operations, an efficient, scalable, and secure relational database system is critical.

Project Objective

To design and implement a fully normalized and relational SQL Server Database for HerculesMotoCorp that ensures:
- Smooth operations for internal users (employees, HR, admin).
- Transparency and accessibility for external stakeholders (e.g., shareholders).
- Legal compliance and auditable transaction records.

Database Design Highlights

- ER Diagram: Created with proper normalization and foreign key relationships.
- Key Tables: customers, employees, orders, orderdetails, products, productlines, payments, offices.
- All entities were logically linked through foreign keys to ensure data integrity and support referential actions.

Key Tasks and Completed Deliverables

Task	Description
ER Diagram	Designed and validated with SSMS including all keys and relationships.
Table Creation	CREATE TABLE scripts generated with proper constraints, indexes, and datatypes.
Data Insertion Order	Defined the correct order of insertion based on table relationships.
Column Clean-up	Removed non-informative columns (e.g., unnecessary fields from productlines).
Insert/Update Verification	Used SELECT statements and joins to confirm all records were

	correctly inserted and updated.
Aggregate Queries	Retrieved highest and lowest amounts from payments, and unique customer counts.
View Operations	Created and dropped view cust_payment for quick data access from customers and payments.
Stored Procedures	Created stored procedure to retrieve products by category (Classic Cars) using parameters.
Functions	Defined a parameterized function to get customers with creditLimit below a threshold.
Triggers	Implemented two triggers: - Insert trigger on employees to log added employees. - Insert trigger on payments to alert if amount > 10,000.
Roles & Security	Created 3 roles (Admin, HR, Employee) with different permission levels. Mapped logins to users.
Access Verification	Verified login-to-role and user-role mappings using queries.

Security Model

- Admin Role: Full access to all objects and operations.
- HR Role: Restricted access to employees and offices only.
- Employee Role: Read-only access to all tables.

Business Value Delivered

- Structured database architecture to reduce redundancy and ensure data consistency.
- User-role-based access control to improve data security.
- Predefined stored procedures, views, and functions to improve maintainability.
- Trigger-based automation for transaction auditing.

Attachments Submitted

- ER Diagram screenshot from SSMS.
- SQL scripts for:

- Table creation
- Stored procedures
- Views
- Triggers
- Functions
- Role/user management
- Data validation queries

Conclusion

This database project for HerculesMotoCorp successfully meets the operational, regulatory, and reporting requirements. The design provides a solid foundation for data-driven decision-making, operational transparency, and future scalability.

Tasks:

1. **Name the database as MotorsCertification. Design an ER model based on the following parameters:**

   Note: Build a ER diagram with proper entities, relationship etc.

   ERDiagram.png

   Based on the ER diagram (which includes tables like products, orderdetails, orders, customers, offices, employees, payments, and productlines), here's the complete SQL Server CREATE TABLE schema that captures all relationships, primary keys, foreign keys, and data types.

   Create Table Schema.sql

```
-- offices
CREATE TABLE offices (
   officeCode VARCHAR(10) PRIMARY KEY,
   city VARCHAR(50),
   phone VARCHAR(50),
   addressLine1 VARCHAR(50),
   addressLine2 VARCHAR(50),
   state VARCHAR(50),
   country VARCHAR(50),
   postalCode VARCHAR(15),
   territory VARCHAR(50)
);
GO

-- employees
CREATE TABLE employees (
   employeeNumber INT PRIMARY KEY,
   lastName VARCHAR(50),
   firstName VARCHAR(50),
   extension VARCHAR(10),
   email VARCHAR(100),
   officeCode VARCHAR(10),
   reportsTo INT,
   jobTitle VARCHAR(50),
   FOREIGN KEY (officeCode) REFERENCES offices(officeCode),
   FOREIGN KEY (reportsTo) REFERENCES employees(employeeNumber)
);
GO
```

```
-- customers
CREATE TABLE customers (
   customerNumber INT PRIMARY KEY,
   customerName VARCHAR(50),
   contactLastName VARCHAR(50),
   contactFirstName VARCHAR(50),
   phone VARCHAR(50),
   addressLine1 VARCHAR(50),
   addressLine2 VARCHAR(50),
   city VARCHAR(50),
   state VARCHAR(50),
   postalCode VARCHAR(15),
   country VARCHAR(50),
   salesRepEmployeeNumber INT,
   creditLimit FLOAT,
   FOREIGN KEY (salesRepEmployeeNumber) REFERENCES
employees(employeeNumber)
);
GO

-- orders
CREATE TABLE orders (
   orderNumber INT PRIMARY KEY,
   orderDate DATE,
   requiredDate DATE,
   shippedDate DATE,
   status VARCHAR(20),
   comments TEXT,
   customerNumber INT,
   FOREIGN KEY (customerNumber) REFERENCES customers(customerNumber)
);
GO

-- productlines
CREATE TABLE productlines (
   productLine VARCHAR(50) PRIMARY KEY,
   textDescription VARCHAR(4000),
   htmlDescription TEXT NULL,
   image VARBINARY(MAX) NULL
);
GO
```

```sql
-- products
CREATE TABLE products (
   productCode VARCHAR(20) PRIMARY KEY,
   productName VARCHAR(100),
   productLine VARCHAR(50),
   productScale VARCHAR(50),
   productVendor VARCHAR(100),
   productDescription TEXT,
   quantityInStock SMALLINT,
   buyPrice FLOAT,
   MSRP FLOAT,
   FOREIGN KEY (productLine) REFERENCES productlines(productLine)
);
GO
-- orderdetails
CREATE TABLE orderdetails (
   orderNumber INT,
   productCode VARCHAR(20),
   quantityOrdered INT,
   priceEach FLOAT,
   orderLineNumber SMALLINT,
   PRIMARY KEY (orderNumber, productCode),
   FOREIGN KEY (orderNumber) REFERENCES orders(orderNumber),
   FOREIGN KEY (productCode) REFERENCES products(productCode)
);
GO

-- payments
CREATE TABLE payments (
   customerNumber INT,
   checkNumber VARCHAR(50),
   paymentDate DATE,
   amount FLOAT,
   PRIMARY KEY (customerNumber, checkNumber),
   FOREIGN KEY (customerNumber) REFERENCES customers(customerNumber)
);
GO
```

Notes:

- Primary Keys are defined on each table.

- Composite Primary Key on orderdetails (orderNumber, productCode).

- All foreign key constraints reflect the ER diagram.
- TEXT and VARBINARY(MAX) used for long text/image columns.

2. After designing the table insert records in the following orderdetails, employees, payments, products, customers, offices and orders table.

To insert records into a relational database while respecting foreign key constraints, we must follow a specific insertion order—starting with the independent (parent) tables first, then dependent (child) tables. Below is the correct step-by-step order to insert records based on schema:

A. **Offices -** No dependencies, fully independent. Insert first.
B. **Employees** - Depends on offices and employees (for reportsTo, which can be NULL initially or refer to existing employee). Insert second.
C. **Productlines** - No dependencies. Insert third.
D. **Products -** Depends on productlines. Insert fourth.
E. **Customers** - Depends on employees (salesRepEmployeeNumber). Insert fifth.
F. **Orders** - Depends on customers.Insert sixth.
G. **Orderdetails** - Depends on orders and products.Insert seventh.
H. **Payments** - Depends on customers. Insert last.

DataTobeinserted.sql

```
use MotorsCertification

--Task 2
--Insert records in customers table
INSERT INTO customers VALUES
	--(103,'Atelier graphique','Schmitt','Carine ','40.32.2555','54, rue Royale',NULL,'Nantes',NULL,'44000','France',1370,'21000.00'),

	(112,'Signal Gift Stores','King','Jean','7025551838','8489 Strong St.',NULL,'Las Vegas','NV','83030','USA',1166,'71800.00'),

	(114,'Australian Collectors, Co.','Ferguson','Peter','03 9520 4555','636 St Kilda Road','Level 3','Melbourne','Victoria','3004','Australia',1611,'117300.00'),

	(119,'La Rochelle Gifts','Labrune','Janine ','40.67.8555','67, rue des Cinquante Otages',NULL,'Nantes',NULL,'44000','France',1370,'118200.00'),

	(121,'Baane Mini Imports','Bergulfsen','Jonas ','07-98 9555','Erling Skakkes gate 78',NULL,'Stavern',NULL,'4110','Norway',1504,'81700.00'),

	(124,'Mini Gifts Distributors Ltd.','Nelson','Susan','4155551450','5677 Strong St.',NULL,'San Rafael','CA','97562','USA',1165,'210500.00'),
```

(125,'Havel & Zbyszek Co','Piestrzeniewicz','Zbyszek ','(26) 642-7555','ul. Filtrowa 68',NULL,'Warszawa',NULL,'01-012','Poland',NULL,'0.00'),

(128,'Blauer See Auto, Co.','Keitel','Roland','+49 69 66 90 2555','Lyonerstr. 34',NULL,'Frankfurt',NULL,'60528','Germany',1504,'59700.00'),

(129,'Mini Wheels Co.','Murphy','Julie','6505555787','5557 North Pendale Street',NULL,'San Francisco','CA','94217','USA',1165,'64600.00'),

(131,'Land of Toys Inc.','Lee','Kwai','2125557818','897 Long Airport Avenue',NULL,'NYC','NY','10022','USA',1323,'114900.00'),

(141,'Euro+ Shopping Channel','Freyre','Diego ','(91) 555 94 44','C/ Moralzarzal, 86',NULL,'Madrid',NULL,'28034','Spain',1370,'227600.00'),

(144,'Volvo Model Replicas, Co','Berglund','Christina ','0921-12 3555','Berguvsvägen 8',NULL,'Luleå',NULL,'S-958 22','Sweden',1504,'53100.00'),

(145,'Danish Wholesale Imports','Petersen','Jytte ','31 12 3555','Vinbæltet 34',NULL,'Kobenhavn',NULL,'1734','Denmark',1401,'83400.00'),

(146,'Saveley & Henriot, Co.','Saveley','Mary ','78.32.5555','2, rue du Commerce',NULL,'Lyon',NULL,'69004','France',1337,'123900.00'),

(148,'Dragon Souveniers, Ltd.','Natividad','Eric','+65 221 7555','Bronz Sok.','Bronz Apt. 3/6 Tesvikiye','Singapore',NULL,'079903','Singapore',1612,'103800.00'),

(151,'Muscle Machine Inc','Young','Jeff','2125557413','4092 Furth Circle','Suite 400','NYC','NY','10022','USA',1286,'138500.00'),

(157,'Diecast Classics Inc.','Leong','Kelvin','2155551555','7586 Pompton St.',NULL,'Allentown','PA','70267','USA',1216,'100600.00'),

(161,'Technics Stores Inc.','Hashimoto','Juri','6505556809','9408 Furth Circle',NULL,'Burlingame','CA','94217','USA',1165,'84600.00'),

(166,'Handji Gifts& Co','Victorino','Wendy','+65 224 1555','106 Linden Road Sandown','2nd Floor','Singapore',NULL,'069045','Singapore',1612,'97900.00'),

```
	(167,'Herkku Gifts','Oeztan','Veysel','+47 2267 3215','Brehmen St.
121','PR 334 Sentrum','Bergen',NULL,'N 5804','Norway  ',1504,'96800.00');

--Validating our records
select * from customers
select * from employees


--Insert records in employees table
insert into employees values

	(1002,'Murphy','Diane','x5800','dmurphy@classicmodelcars.com','1',NU
LL,'President'),

	(1056,'Patterson','Mary','x4611','mpatterso@classicmodelcars.com','1',
1002,'VP Sales'),

	(1076,'Firrelli','Jeff','x9273','jfirrelli@classicmodelcars.com','1',1002,'VP
Marketing'),

	(1088,'Patterson','William','x4871','wpatterson@classicmodelcars.com',
'6',1056,'Sales Manager (APAC)'),

	(1102,'Bondur','Gerard','x5408','gbondur@classicmodelcars.com','4',10
56,'Sale Manager (EMEA)'),

	(1143,'Bow','Anthony','x5428','abow@classicmodelcars.com','1',1056,'S
ales Manager (NA)'),

	(1165,'Jennings','Leslie','x3291','ljennings@classicmodelcars.com','1',11
43,'Sales Rep'),

	(1166,'Thompson','Leslie','x4065','lthompson@classicmodelcars.com','1
',1143,'Sales Rep'),

	(1188,'Firrelli','Julie','x2173','jfirrelli@classicmodelcars.com','2',1143,'S
ales Rep'),

	(1216,'Patterson','Steve','x4334','spatterson@classicmodelcars.com','2',
1143,'Sales Rep'),
```

(1286,'Tseng','Foon Yue','x2248','ftseng@classicmodelcars.com','3',1143,'Sales Rep'),

(1323,'Vanauf','George','x4102','gvanauf@classicmodelcars.com','3',1143,'Sales Rep'),

(1337,'Bondur','Loui','x6493','lbondur@classicmodelcars.com','4',1102,'Sales Rep'),

(1370,'Hernandez','Gerard','x2028','ghernande@classicmodelcars.com','4',1102,'Sales Rep'),

(1401,'Castillo','Pamela','x2759','pcastillo@classicmodelcars.com','4',1102,'Sales Rep'),

(1501,'Bott','Larry','x2311','lbott@classicmodelcars.com','7',1102,'Sales Rep'),

(1504,'Jones','Barry','x102','bjones@classicmodelcars.com','7',1102,'Sales Rep'),

(1611,'Fixter','Andy','x101','afixter@classicmodelcars.com','6',1088,'Sales Rep'),

(1612,'Marsh','Peter','x102','pmarsh@classicmodelcars.com','6',1088,'Sales Rep'),

(1619,'King','Tom','x103','tking@classicmodelcars.com','6',1088,'Sales Rep');

select * from employees

insert into offices values

('1','San Francisco','+1 650 219 4782','100 Market Street','Suite 300','CA','USA','94080','NA'),

('2','Boston','+1 215 837 0825','1550 Court Place','Suite 102','MA','USA','02107','NA'),

('3','NYC','+1 212 555 3000','523 East 53rd Street','apt. 5A','NY','USA','10022','NA'),

('4','Paris','+33 14 723 4404','43 Rue Jouffroy Dabbans',NULL,NULL,'France','75017','EMEA'),

('5','Tokyo','+81 33 224 5000','4-1 Kioicho',NULL,'Chiyoda-Ku','Japan','102-8578','Japan'),

('6','Sydney','+61 2 9264 2451','5-11 Wentworth Avenue','Floor #2',NULL,'Australia','NSW 2010','APAC'),

('7','London','+44 20 7877 2041','25 Old Broad Street','Level 7',NULL,'UK','EC2N 1HN','EMEA');

select * from offices

insert into orderdetails values

(10100,'S10_1678',30,'136.00',3),

(10101,'S10_4698',50,'55.09',2),

(10102,'S18_1129',22,'75.46',4),

(10103,'S12_1108',49,'35.29',1),

(10104,'S10_4757',25,'108.06',4),

(10105,'S18_1129',26,'167.06',1),

(10106,'S12_1666',45,'32.53',3),

(10107,'S12_3891',46,'44.35',2),

(10108,'S18_1342',39,'95.55',2),

(10109,'S18_1367',41,'43.13',1),

(10110,'S10_1949',26,'214.30',11),

(10111,'S10_4962',42,'119.67',4),

(10112,'S12_1666',27,'121.64',8),

(10113,'S18_1097',35,'94.50',10),

(10114,'S12_4675',22,'58.34',2),

(10115,'S12_2823',27,'92.19',12),

(10116,'S10_2016',35,'61.84',14),

(10117,'S12_3148',25,'86.92',13),

(10118,'S12_3380',46,'86.31',16),

(10119,'S12_4473',36,'98.07',5);

use MotorsCertification

select * from products
select * from orderdetails
select * from orders
insert into orders values
(10100,'2003-01-06','2003-01-13','2003-01-10','Shipped',NULL,103),

(10101,'2003-01-09','2003-01-18','2003-01-11','Shipped','Check on availability.',128),

(10102,'2003-01-10','2003-01-18','2003-01-14','Shipped',NULL,151),

(10103,'2003-01-29','2003-02-07','2003-02-02','Shipped',NULL,121),

(10104,'2003-01-31','2003-02-09','2003-02-01','Shipped',NULL,141),

(10105,'2003-02-11','2003-02-21','2003-02-12','Shipped',NULL,145),

(10106,'2003-02-17','2003-02-24','2003-02-21','Shipped',NULL,148),

(10107,'2003-02-24','2003-03-03','2003-02-26','Shipped','Difficult to negotiate with customer. We need more marketing materials',131),

(10108,'2003-03-03','2003-03-12','2003-03-08','Shipped',NULL,125),

(10109,'2003-03-10','2003-03-19','2003-03-11','Shipped','Customer requested that FedEx Ground is used for this shipping',166),

(10110,'2003-03-18','2003-03-24','2003-03-20','Shipped',NULL,167),

(10111,'2003-03-25','2003-03-31','2003-03-30','Shipped',NULL,129),

(10112,'2003-03-24','2003-04-03','2003-03-29','Shipped','Customer requested that ad materials (such as posters, pamphlets) be included in the shippment',144),

(10113,'2003-03-26','2003-04-02','2003-03-27','Shipped',NULL,124),

(10114,'2003-04-01','2003-04-07','2003-04-02','Shipped',NULL,157),

(10115,'2003-04-04','2003-04-12','2003-04-07','Shipped',NULL,114),

(10116,'2003-04-11','2003-04-19','2003-04-13','Shipped',NULL,146),

(10117,'2003-04-16','2003-04-24','2003-04-17','Shipped',NULL,148),

(10118,'2003-04-21','2003-04-29','2003-04-26','Shipped','Customer has worked with some of our vendors in the past and is aware of their MSRP',167),

(10119,'2003-04-28','2003-05-05','2003-05-02','Shipped',NULL,128);
select * from customers
select * from employees
insert into payments values
(121,'HQ336336','2004-10-19','6066.78'),

(112,'JM555205','2003-06-05','14571.44'),

(103,'OM314933','2004-12-18','1676.14'),

(114,'BO864823','2004-12-17','14191.12'),

(125,'HQ55022','2003-06-06','32641.98'),

(146,'ND748579','2004-08-20','33347.88'),

(157,'GG31455','2003-05-20','45864.03'),

(128,'MA765515','2004-12-15','82261.22'),

(119,'NP603840','2003-05-31','7565.08'),

(141,'NR27552','2004-03-10','44894.74'),

(131,'DB933704','2004-11-14','19501.82'),

(112,'LN373447','2004-08-08','47924.19'),

(144,'NG94694','2005-02-22','49523.67'),

(114,'DB889831','2003-02-16','50218.95'),

(125,'FD317790','2003-10-28','1491.38'),

(146,'KI831359','2004-11-04','17876.32'),

(157,'MA302151','2004-11-28','34638.14'),

(128,'AE215433','2005-03-05','101244.59'),

(119,'BG255406','2004-08-28','85410.87'),

(151,'CQ287967','2003-04-11','11044.30');

select * from payments
insert into productlines values
('Classic Cars','Attention car enthusiasts: Make your wildest car ownership dreams come true. Whether you are looking for classic muscle cars, dream sports cars or movie-inspired miniatures, you will find great choices in this category. These replicas feature superb attention to detail and craftsmanship and offer features such as working steering system, opening forward compartment, opening rear trunk with removable spare wheel, 4-wheel independent spring suspension, and so on. The models range in size from 1:10 to 1:24 scale and include numerous limited edition and several out-of-production vehicles. All models include a certificate of authenticity from their manufacturers and come fully assembled and ready for display in the home or office.',NULL,NULL),

('Motorcycles','Our motorcycles are state of the art replicas of classic as well as contemporary motorcycle legends such as Harley Davidson, Ducati and Vespa. Models contain stunning details such as official logos, rotating wheels, working kickstand, front suspension, gear-shift lever, footbrake lever, and drive chain. Materials used include diecast and plastic. The models range in size from 1:10 to 1:50 scale and include numerous limited edition and several

out-of-production vehicles. All models come fully assembled and ready for display in the home or office. Most include a certificate of authenticity.',NULL,NULL),

('Planes','Unique, diecast airplane and helicopter replicas suitable for collections, as well as home, office or classroom decorations. Models contain stunning details such as official logos and insignias, rotating jet engines and propellers, retractable wheels, and so on. Most come fully assembled and with a certificate of authenticity from their manufacturers.',NULL,NULL),

('Ships','The perfect holiday or anniversary gift for executives, clients, friends, and family. These handcrafted model ships are unique, stunning works of art that will be treasured for generations! They come fully assembled and ready for display in the home or office. We guarantee the highest quality, and best value.',NULL,NULL),

('Trains','Model trains are a rewarding hobby for enthusiasts of all ages. Whether you are looking for collectible wooden trains, electric streetcars or locomotives, you will find a number of great choices for any budget within this category. The interactive aspect of trains makes toy trains perfect for young children. The wooden train sets are ideal for children under the age of 5.',NULL,NULL),

('Trucks and Buses','The Truck and Bus models are realistic replicas of buses and specialized trucks produced from the early 1920s to present. The models range in size from 1:12 to 1:50 scale and include numerous limited edition and several out-of-production vehicles. Materials used include tin, diecast and plastic. All models include a certificate of authenticity from their manufacturers and are a perfect ornament for the home and office.',NULL,NULL),

('Vintage Cars','Our Vintage Car models realistically portray automobiles produced from the early 1900s through the 1940s. Materials used include Bakelite, diecast, plastic and wood. Most of the replicas are in the 1:18 and 1:24 scale sizes, which provide the optimum in detail and accuracy. Prices range from $30.00 up to $180.00 for some special limited edition replicas. All models include a certificate of authenticity from their manufacturers and come fully assembled and ready for display in the home or office.',NULL,NULL);

select * from productlines
---Inserting into products table

insert into products values

('S10_1678','1969 Harley Davidson Ultimate Chopper','Motorcycles','1:10','Min Lin Diecast','This replica features working kickstand, front suspension, gear-shift lever, footbrake lever, drive chain, wheels and steering. All parts are particularly delicate due to their precise scale and require special care and attention.',7933,'48.81','95.70'),

('S10_1949','1952 Alpine Renault 1300','Classic Cars','1:10','Classic Metal Creations','Turnable front wheels; steering function; detailed interior; detailed engine; opening hood; opening trunk; opening doors; and detailed chassis.',7305,'98.58','214.30'),

('S10_2016','1996 Moto Guzzi 1100i','Motorcycles','1:10','Highway 66 Mini Classics','Official Moto Guzzi logos and insignias, saddle bags located on side of motorcycle, detailed engine, working steering, working suspension, two leather seats, luggage rack, dual exhaust pipes, small saddle bag located on handle bars, two-tone paint with chrome accents, superior die-cast detail , rotating wheels , working kick stand, diecast metal with plastic parts and baked enamel finish.',6625,'68.99','118.94'),

('S10_4698','2003 Harley-Davidson Eagle Drag Bike','Motorcycles','1:10','Red Start Diecast','Model features, official Harley Davidson logos and insignias, detachable rear wheelie bar, heavy diecast metal with resin parts, authentic multi-color tampo-printed graphics, separate engine drive belts, free-turning front fork, rotating tires and rear racing slick, certificate of authenticity, detailed engine, display stand\r\n, precision diecast replica, baked enamel finish, 1:10 scale model, removable fender, seat and tank cover piece for displaying the superior detail of the v-twin engine',5582,'91.02','193.66'),

('S10_4757','1972 Alfa Romeo GTA','Classic Cars','1:10','Motor City Art Classics','Features include: Turnable front wheels; steering function; detailed interior; detailed engine; opening hood; opening trunk; opening doors; and detailed chassis.',3252,'85.68','136.00'),

('S10_4962','1962 LanciaA Delta 16V','Classic Cars','1:10','Second Gear Diecast','Features include: Turnable front wheels; steering function; detailed interior; detailed engine; opening hood; opening trunk; opening doors; and detailed chassis.',6791,'103.42','147.74'),

('S12_1099','1968 Ford Mustang','Classic Cars','1:12','Autoart Studio Design','Hood, doors and trunk all open to reveal highly detailed interior

features. Steering wheel actually turns the front wheels. Color dark green.',68,'95.34','194.57'),

('S12_1108','2001 Ferrari Enzo','Classic Cars','1:12','Second Gear Diecast','Turnable front wheels; steering function; detailed interior; detailed engine; opening hood; opening trunk; opening doors; and detailed chassis.',3619,'95.59','207.80'),

('S12_1666','1958 Setra Bus','Trucks and Buses','1:12','Welly Diecast Productions','Model features 30 windows, skylights & glare resistant glass, working steering system, original logos',1579,'77.90','136.67'),

('S12_2823','2002 Suzuki XREO','Motorcycles','1:12','Unimax Art Galleries','Official logos and insignias, saddle bags located on side of motorcycle, detailed engine, working steering, working suspension, two leather seats, luggage rack, dual exhaust pipes, small saddle bag located on handle bars, two-tone paint with chrome accents, superior die-cast detail , rotating wheels , working kick stand, diecast metal with plastic parts and baked enamel finish.',9997,'66.27','150.62'),

('S12_3148','1969 Corvair Monza','Classic Cars','1:18','Welly Diecast Productions','1:18 scale die-cast about 10\" long doors open, hood opens, trunk opens and wheels roll',6906,'89.14','151.08'),

('S12_3380','1968 Dodge Charger','Classic Cars','1:12','Welly Diecast Productions','1:12 scale model of a 1968 Dodge Charger. Hood, doors and trunk all open to reveal highly detailed interior features. Steering wheel actually turns the front wheels. Color black',9123,'75.16','117.44'),

('S12_3891','1969 Ford Falcon','Classic Cars','1:12','Second Gear Diecast','Turnable front wheels; steering function; detailed interior; detailed engine; opening hood; opening trunk; opening doors; and detailed chassis.',1049,'83.05','173.02'),

('S12_3990','1970 Plymouth Hemi Cuda','Classic Cars','1:12','Studio M Art Models','Very detailed 1970 Plymouth Cuda model in 1:12 scale. The Cuda is generally accepted as one of the fastest original muscle cars from the 1970s. This model is a reproduction of one of the orginal 652 cars built in 1970. Red color.',5663,'31.92','79.80'),

('S12_4473','1957 Chevy Pickup','Trucks and Buses','1:12','Exoto Designs','1:12 scale die-cast about 20\" long Hood opens, Rubber wheels',6125,'55.70','118.50'),

('S12_4675','1969 Dodge Charger','Classic Cars','1:12','Welly Diecast Productions','Detailed model of the 1969 Dodge Charger. This model includes finely detailed interior and exterior features. Painted in red and white.',7323,'58.73','115.16'),

('S18_1097','1940 Ford Pickup Truck','Trucks and Buses','1:18','Studio M Art Models','This model features soft rubber tires, working steering, rubber mud guards, authentic Ford logos, detailed undercarriage, opening doors and hood, removable split rear gate, full size spare mounted in bed, detailed interior with opening glove box',2613,'58.33','116.67'),

('S18_1129','1993 Mazda RX-7','Classic Cars','1:18','Highway 66 Mini Classics','This model features, opening hood, opening doors, detailed engine, rear spoiler, opening trunk, working steering, tinted windows, baked enamel finish. Color red.',3975,'83.51','141.54'),

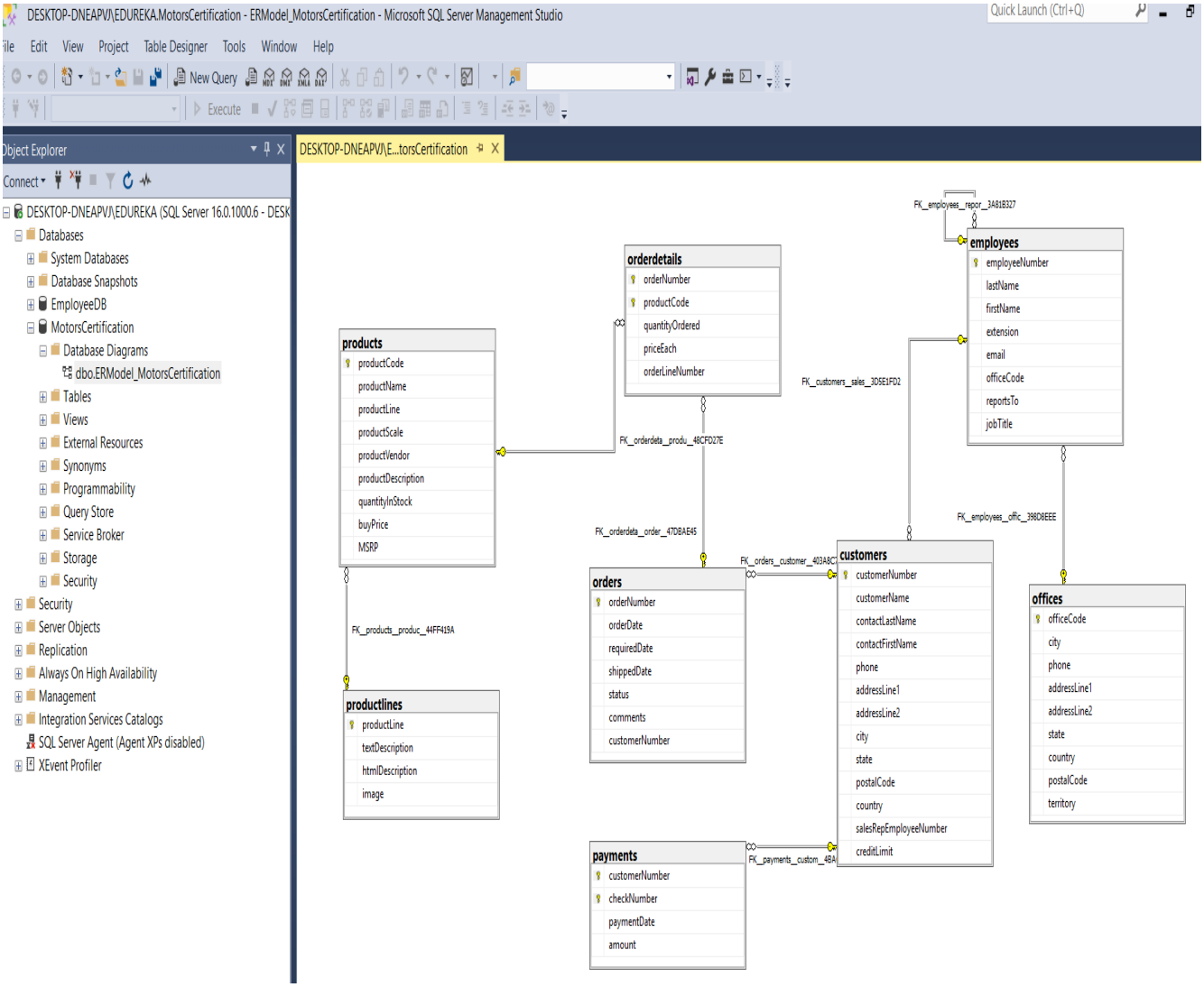
('S18_1342','1937 Lincoln Berline','Vintage Cars','1:18','Motor City Art Classics','Features opening engine cover, doors, trunk, and fuel filler cap. Color black',8693,'60.62','102.74'),

('S18_1367','1936 Mercedes-Benz 500K Special Roadster','Vintage Cars','1:18','Studio M Art Models','This 1:18 scale replica is constructed of heavy die-cast metal and has all the features of the original: working doors and rumble seat, independent spring suspension, detailed interior, working steering system, and a bifold hood that reveals an engine so accurate that it even includes the wiring. All this is topped off with a baked enamel finish. Color white.',8635,'24.26','53.91');

select * from products

select * from productlines
select * from products
select * from payments
select * from offices
select * from orderdetails
select * from orders
select * from customers
select * from employees

3. Provide comments before every task that is performed describing the operation that is being performed and attach a screenshot of ER diagram from SSMS.



This ER diagram models a classic business scenario for **orders, customers, employees, and products**. It includes **8 entities** and their relationships.

◆ 1. products

- **Primary Key:** productCode
- Connected to productlines and orderdetails
- **Attributes include:** productName, productLine, buyPrice, MSRP
- **Relationships:**
  - **Many-to-One** with productlines (via productLine)
  - **One-to-Many** with orderdetails (via productCode)

◆ 2. productlines

- **Primary Key:** productLine
- Holds higher-level grouping of products (e.g., Classic Cars, Motorcycles)
- **Attributes include:** textDescription, htmlDescription
- **Relationship:**
  - **One-to-Many** with products

◆ 3. orderdetails

- **Composite Foreign Key:** orderNumber and productCode
- Contains details of each item in an order
- **Attributes include:** quantityOrdered, priceEach, orderLineNumber
- **Relationships:**
  - **Many-to-One** with orders (via orderNumber)
  - **Many-to-One** with products (via productCode)

◆ 4. orders

- **Primary Key:** orderNumber
- Represents customer orders
- **Attributes include:** orderDate, requiredDate, status, customerNumber
- **Relationship:**
  - **Many-to-One** with customers (via customerNumber)
  - **One-to-Many** with orderdetails

◆ 5. customers

- **Primary Key:** customerNumber
- Stores customer information
- **Attributes include:** customerName, contactFirstName, phone, creditLimit, etc.
- **Relationships:**
  - **One-to-Many** with orders
  - **One-to-Many** with payments
  - **Many-to-One** with employees (via salesRepEmployeeNumber)

◆ 6. payments

- **Composite Primary Key:** customerNumber, checkNumber
- Stores payment information made by customers
- **Attributes include:** paymentDate, amount
- **Relationship:**
  - **Many-to-One** with customers

◆ 7. employees

- **Primary Key:** employeeNumber
- Represents company staff
- **Attributes include:** lastName, email, jobTitle, etc.
- **Self-Referential FK:** reportsTo → employeeNumber
- **Relationships:**
  - **One-to-Many** to self (managers to subordinates)
  - **Many-to-One** with offices (via officeCode)
  - **One-to-Many** with customers (as sales reps)

◆ 8. offices

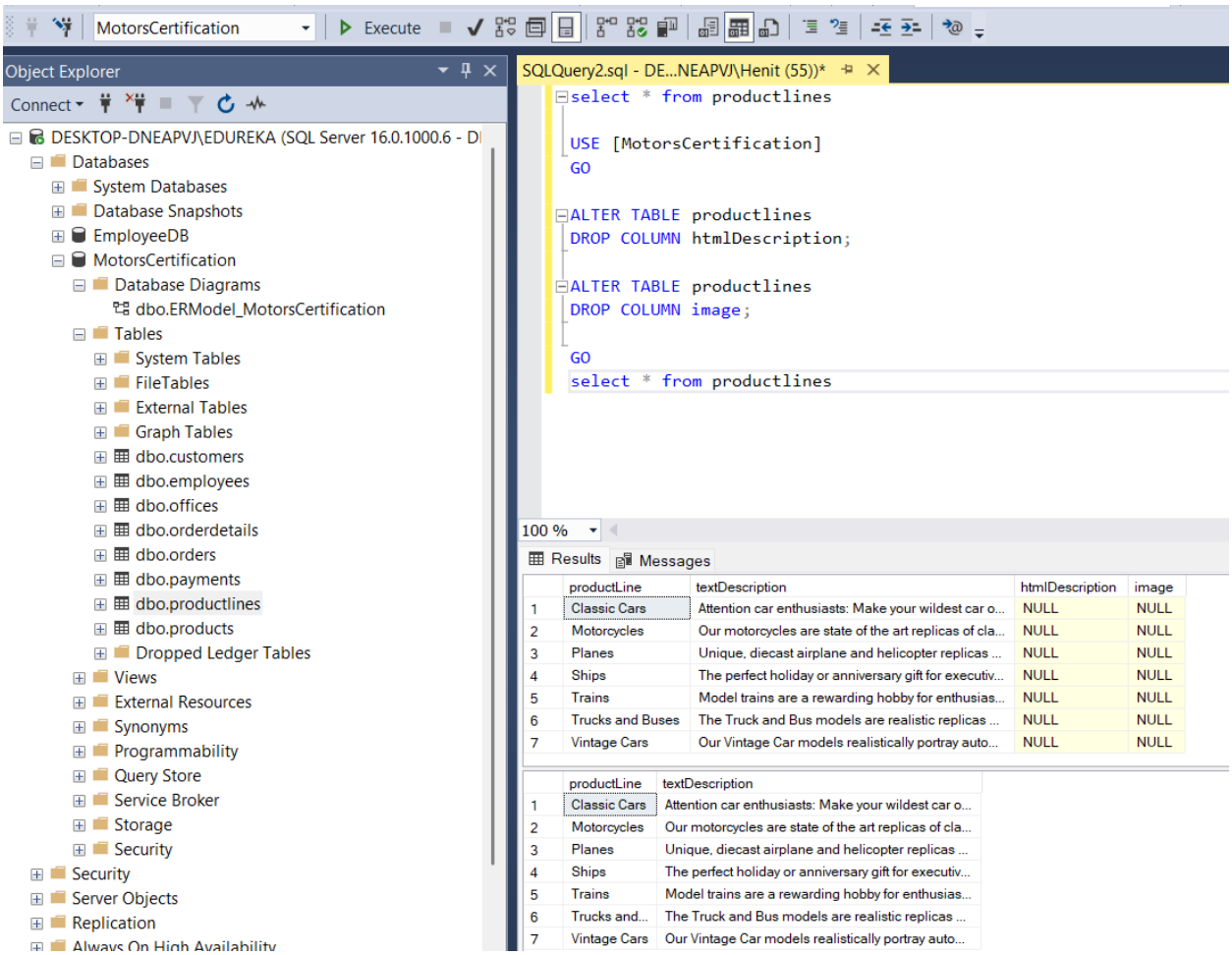
- **Primary Key:** officeCode
- Stores office location details
- **Attributes include:** city, state, territory, etc.
- **Relationship:**
  - **One-to-Many** with employees

4. Delete the columns in productlines which are useless that do not infer anything.

Columns to Remove:

- htmlDescription: Usually used for web rendering, not useful for analytics or transactions.

- image: Binary data that doesn't contribute to inference/relationships, usually stored separately in content management.



5. Use a select statement to verify all insertions as well as updates.

To verify all insertions and updates made in our database tables, we can run SELECT statements for each table. Here's a complete set of SELECT statements to check the data in each table:

1. Verify inserted or updated rows in 'productlines'
SELECT * FROM productlines;

2. Verify inserted or updated rows in 'products'
SELECT * FROM products;

3. Verify inserted or updated rows in 'orderdetails'
SELECT * FROM orderdetails;

4. Verify inserted or updated rows in 'orders'
SELECT * FROM orders;

5. Verify inserted or updated rows in 'customers'
SELECT * FROM customers;

6. Verify inserted or updated rows in 'employees'
SELECT * FROM employees;

7. Verify inserted or updated rows in 'offices'
SELECT * FROM offices;

8. Verify inserted or updated rows in 'payments'
SELECT * FROM payments;

Full Relationship Chain Example

SQLQuery1.sql - DE...NEAPVJ\Henit (58))*

```sql
SELECT
    o.orderNumber,
    c.customerName,
    e.firstName AS salesRepFirstName,
    od.productCode,
    pr.productName,
    pr.productLine,
    od.quantityOrdered,
    od.priceEach
FROM orders o
JOIN customers c ON o.customerNumber = c.customerNumber
JOIN employees e ON c.salesRepEmployeeNumber = e.employeeNumber
JOIN orderdetails od ON o.orderNumber = od.orderNumber
JOIN products pr ON od.productCode = pr.productCode;
```

100 %

Results | Messages

	orderNumber	customerName	salesRepFirstName	productCode	productName	productLine	quantityOrdered	priceEach
1	10100	Atelier graphique	Gerard	S10_1678	1969 Harley Davidson Ultimate Chopper	Motorcycles	30	136
2	10101	Blauer See Auto, Co.	Barry	S10_4698	2003 Harley-Davidson Eagle Drag Bike	Motorcycles	50	55.09
3	10102	Muscle Machine Inc	Foon Yue	S18_1129	1993 Mazda RX-7	Classic Cars	22	75.46
4	10103	Baane Mini Imports	Barry	S12_1108	2001 Ferrari Enzo	Classic Cars	49	35.29
5	10104	Euro+ Shopping Channel	Gerard	S10_4757	1972 Alfa Romeo GTA	Classic Cars	25	108.06
6	10105	Danish Wholesale Imports	Pamela	S18_1129	1993 Mazda RX-7	Classic Cars	26	167.06
7	10106	Dragon Souveniers, Ltd.	Peter	S12_1666	1958 Setra Bus	Trucks and Buses	45	32.53
8	10107	Land of Toys Inc.	George	S12_3891	1969 Ford Falcon	Classic Cars	46	44.35
9	10109	Handji Gifts& Co	Peter	S18_1367	1936 Mercedes-Benz 500K Special Roadster	Vintage Cars	41	43.13
10	10110	Herkku Gifts	Barry	S10_1949	1952 Alpine Renault 1300	Classic Cars	26	214.3
11	10111	Mini Wheels Co.	Leslie	S10_4962	1962 LanciaA Delta 16V	Classic Cars	42	119.67
12	10112	Volvo Model Replicas, Co	Barry	S12_1666	1958 Setra Bus	Trucks and Buses	27	121.64
13	10113	Mini Gifts Distributors Ltd.	Leslie	S18_1097	1940 Ford Pickup Truck	Trucks and Buses	35	94.5
14	10114	Diecast Classics Inc.	Steve	S12_4675	1969 Dodge Charger	Classic Cars	22	58.34
15	10115	Australian Collectors, Co.	Andy	S12_2823	2002 Suzuki XREO	Motorcycles	27	92.19
16	10116	Saveley & Henriot, Co.	Loui	S10_2016	1996 Moto Guzzi 1100i	Motorcycles	35	61.84

6. Find out the highest and the lowest amount.

Explanation:

- MAX(amount) returns the highest payment in the amount column.
- MIN(amount) returns the lowest payment in the amount column.
- The result will return one row with two columns: HighestAmount and LowestAmount.

```sql
SELECT
    MAX(amount) AS HighestAmount,
    MIN(amount) AS LowestAmount
FROM payments
```

100 %

Results | Messages

	HighestAmount	LowestAmount
1	101244.59	1491.38

Also Showing Customer Info Along With Those Values:

```sql
SELECT
    MAX(amount) AS HighestAmount,
    MIN(amount) AS LowestAmount
FROM payments

-- Highest Payment
SELECT TOP 1 p.customerNumber, c.customerName, p.checkNumber, p.paymentDate, p.amount
FROM payments p
JOIN customers c ON p.customerNumber = c.customerNumber
ORDER BY p.amount DESC;

-- Lowest Payment
SELECT TOP 1 p.customerNumber, c.customerName, p.checkNumber, p.paymentDate, p.amount
FROM payments p
JOIN customers c ON p.customerNumber = c.customerNumber
ORDER BY p.amount ASC;
```

100 %

Results | Messages

	HighestAmount	LowestAmount
1	101244.59	1491.38

	customerNumber	customerName	checkNumber	paymentDate	amount
1	128	Blauer See Auto, Co.	AE215433	2005-03-05	101244.59

	customerNumber	customerName	checkNumber	paymentDate	amount
1	125	Havel & Zbyszek Co	FD317790	2003-10-28	1491.38

7. Give the unique count of customerName from customers.

Explanation:

- COUNT(DISTINCT customerName) counts how many distinct (non-duplicate) customer names exist in the customers table.

- AS uniqueCustomerCount is an alias for better readability of the output column.

```
SELECT COUNT(DISTINCT customerName) AS UniqueCustomerCount
FROM customers
```

100 %

Results | Messages

	customerNumber	customerName	contactLastName	contactFirstName	phone	addressLine1	addressLine2	city	state	postalCode	country	salesRepEmployeeNumbe
1	103	Atelier graphique	Schmitt	Carine	40.32.2555	54, rue Royale	NULL	Nantes	NULL	44000	France	1370
2	112	Signal Gift Stores	King	Jean	7025551838	8489 Strong St.	NULL	Las Vegas	NV	83030	USA	1166
3	114	Australian Collectors, Co.	Ferguson	Peter	03 9520 4555	636 St Kilda Road	Level 3	Melbourne	Victoria	3004	Australia	1611
4	119	La Rochelle Gifts	Labrune	Janine	40.67.8555	67, rue des Cinquante Otages	NULL	Nantes	NULL	44000	France	1370
5	121	Baane Mini Imports	Bergulfsen	Jonas	07-98 9555	Erling Skakkes gate 78	NULL	Stavern	NULL	4110	Norway	1504
6	124	Mini Gifts Distributors Ltd.	Nelson	Susan	4155551450	5677 Strong St.	NULL	San Rafael	CA	97562	USA	1165
7	125	Havel & Zbyszek Co	Piestrzeniewicz	Zbyszek	(26) 642-7555	ul. Filtrowa 68	NULL	Warszawa	NULL	01-012	Poland	NULL
8	128	Blauer See Auto, Co.	Keitel	Roland	+49 69 66 90 2555	Lyonerstr. 34	NULL	Frankfurt	NULL	60528	Germany	1504
9	129	Mini Wheels Co.	Murphy	Julie	6505555787	5557 North Pendale Street	NULL	San Francisco	CA	94217	USA	1165
10	131	Land of Toys Inc.	Lee	Kwai	2125557818	897 Long Airport Avenue	NULL	NYC	NY	10022	USA	1323
11	141	Euro+ Shopping Channel	Freyre	Diego	(91) 555 94 44	C/ Moralzarzal, 86	NULL	Madrid	NULL	28034	Spain	1370
12	144	Volvo Model Replicas, Co	Berglund	Christina	0921-12 3555	Berguvsvägen 8	NULL	Luleå	NULL	S-958 22	Sweden	1504
13	145	Danish Wholesale Imports	Petersen	Jytte	31 12 3555	Vinbæltet 34	NULL	Kobenhavn	NULL	1734	Denmark	1401
14	146	Saveley & Henriot, Co.	Saveley	Mary	78.32.5555	2, rue du Commerce	NULL	Lyon	NULL	69004	France	1337
15	148	Dragon Souveniers, Ltd.	Natividad	Eric	+65 221 7555	Bronz Sok.	Bronz Apt. 3/6 Tesvikiye	Singapore	NULL	079903	Singapore	1612
16	151	Muscle Machine Inc	Young	Jeff	2125557413	4092 Furth Circle	Suite 400	NYC	NY	10022	USA	1286
17	157	Diecast Classics Inc.	Leong	Kelvin	2155551555	7586 Pompton St.	NULL	Allentown	PA	70267	USA	1216
18	161	Technics Stores Inc.	Hashimoto	Juri	6505556809	9408 Furth Circle	NULL	Burlingame	CA	94217	USA	1165
19	166	Handji Gifts& Co	Victorino	Wendy	+65 224 1555	106 Linden Road Sandown	2nd Floor	Singapore	NULL	069045	Singapore	1612
20	167	Herkku Gifts	Oeztan	Veysel	+47 2267 3215	Brehmen St. 121	PR 334 Sentrum	Bergen	NULL	N 5804	Norway	1504

Query executed successfully. | DESKTOP-DNEAPVJ\EDUREKA (16... | DESKTOP-DNEAPVJ\Henit ... | MotorsCertification | 00:00:00 | 20 rows

8. Create a view from customers and payments named cust_payment and select customerName, amount, contactLastName, contactFirstName who have paid. Truncate and Drop the view after operation.

Step-by-Step Explanation:

1. **Create View cust_payment**:
   - The view will **join** the customers and payments tables using customerNumber.
   - It will **select**:
     - customerName
     - amount
     - contactLastName
     - contactFirstName
   - Only customers who have made payments will appear in the view.
2. **Use the View**:
   - Execute a SELECT * FROM cust_payment to verify the output.
3. **Truncate the View**:
   - **Note**: We **cannot truncate** a view directly in SQL Server. Views are **virtual tables**, not physical ones. If we need to truncate underlying data, we must truncate the base table (not advisable unless it's safe).
4. **Drop the View**:
   - Use DROP VIEW cust_payment; to remove the view from the database after we are done using it.

Create View:

VIEW cust_payment...NEAPVJ\Henit (68))* | SQLQuery1.sql - not connected

```sql
CREATE VIEW cust_payment AS
SELECT
    c.customerName,
    p.amount,
    c.contactLastName,
    c.contactFirstName
FROM customers c
JOIN payments p ON c.customerNumber = p.customerNumber

GO
```

100 %

Messages

```
Commands completed successfully.

Completion time: 2025-05-14T19:43:48.2234737-05:00
```

Select the view /Run the view

```sql
select * from cust_payment
GO

TRUNCATE table cust_payment

GO
```

100 %

Results | Messages

	customerName	amount	contactLastName	contactFirstName
1	Atelier graphique	1676.14	Schmitt	Carine
2	Signal Gift Stores	14571.44	King	Jean
3	Signal Gift Stores	47924.19	King	Jean
4	Australian Collectors, Co.	14191.12	Ferguson	Peter
5	Australian Collectors, Co.	50218.95	Ferguson	Peter
6	La Rochelle Gifts	85410.87	Labrune	Janine
7	La Rochelle Gifts	7565.08	Labrune	Janine
8	Baane Mini Imports	6066.78	Bergulfsen	Jonas
9	Havel & Zbyszek Co	1491.38	Piestrzeniewicz	Zbyszek
10	Havel & Zbyszek Co	32641.98	Piestrzeniewicz	Zbyszek
11	Blauer See Auto, Co.	101244.59	Keitel	Roland
12	Blauer See Auto, Co.	82261.22	Keitel	Roland
13	Land of Toys Inc.	19501.82	Lee	Kwai
14	Euro+ Shopping Channel	44894.74	Freyre	Diego
15	Volvo Model Replicas, Co	49523.67	Berglund	Christina
16	Saveley & Henriot, Co.	17876.32	Saveley	Mary

Truncate view command: Error

VIEW cust_payment...NEAPVJ\Henit (68))* | SQLQuery1.sql - not connected

```sql
GO

TRUNCATE table cust_payment

GO
```

100 %

Messages

```
Msg 4708, Level 16, State 2, Line 15
Could not truncate object 'cust_payment' because it is not a table.

Completion time: 2025-05-14T19:52:41.8913920-05:00
```

Drop View:

VIEW cust_payment...NEAPVJ\Henit (68))* | SQLQuery1.sql - not co

```sql
DROP VIEW cust_payment
```

100 %

Messages

```
Commands completed successfully.

Completion time: 2025-05-14T19:53:13.9568064-05:00
```

9. Create a stored procedure on products which displays productLine for Classic Cars.

Create a stored procedure that returns all products in a given productLine
The procedure takes a category name (e.g., 'Classic Cars') as input

SP_GetProductsByC...NEAPVJ\Henit (53))* | SQLQuery1.sql - not connected

```sql
CREATE PROCEDURE GetProductsByCategory
    @CategoryName VARCHAR(50)
AS
BEGIN
    SELECT
        productCode,
        productName,
        productLine,
        productScale,
        productVendor,
        quantityInStock,
        buyPrice,
        MSRP
    FROM products
    WHERE productLine = @CategoryName

END
```

100 %

Messages

```
Commands completed successfully.

Completion time: 2025-05-14T19:55:58.7152272-05:00
```

Execute the procedure to return all products in 'Classic Cars' category

```
EXEC GetProductsByCategory @CategoryName = 'Classic Cars'

EXEC GetProductsByCategory @CategoryName = 'Vintage Cars'
```

100 %

Results | Messages

	productCode	productName	productLine	productScale	productVendor	quantityInStock	buyPrice	MSRP
1	S10_1949	1952 Alpine Renault 1300	Classic Cars	1:10	Classic Metal Creations	7305	98.58	214.3
2	S10_4757	1972 Alfa Romeo GTA	Classic Cars	1:10	Motor City Art Classics	3252	85.68	136
3	S10_4962	1962 LanciaA Delta 16V	Classic Cars	1:10	Second Gear Diecast	6791	103.42	147.74
4	S12_1099	1968 Ford Mustang	Classic Cars	1:12	Autoart Studio Design	68	95.34	194.57
5	S12_1108	2001 Ferrari Enzo	Classic Cars	1:12	Second Gear Diecast	3619	95.59	207.8
6	S12_3148	1969 Corvair Monza	Classic Cars	1:18	Welly Diecast Productions	6906	89.14	151.08
7	S12_3380	1968 Dodge Charger	Classic Cars	1:12	Welly Diecast Productions	9123	75.16	117.44
8	S12_3891	1969 Ford Falcon	Classic Cars	1:12	Second Gear Diecast	1049	83.05	173.02
9	S12_3990	1970 Plymouth Hemi Cuda	Classic Cars	1:12	Studio M Art Models	5663	31.92	79.8
10	S12_4675	1969 Dodge Charger	Classic Cars	1:12	Welly Diecast Productions	7323	58.73	115.16
11	S18_1129	1993 Mazda RX-7	Classic Cars	1:18	Highway 66 Mini Classics	3975	83.51	141.54

Execute the procedure to test with another product line like 'Vintage Cars'
This ensures reusability of the stored procedure

```
-- 2. Execute the procedure to return all products in 'Classic Cars' category
EXEC GetProductsByCategory @CategoryName = 'Classic Cars';
GO

-- 3. Execute the procedure to test with another product line like 'Vintage Cars'
-- This ensures reusability of the stored procedure
EXEC GetProductsByCategory @CategoryName = 'Vintage Cars';
GO
```

100 %

Results | Messages

	productCode	productName	productLine	productScale	productVendor	quantityInStock	buyPrice	MSRP
1	S18_1342	1937 Lincoln Berline	Vintage Cars	1:18	Motor City Art Classics	8693	60.62	102.74
2	S18_1367	1936 Mercedes-Benz 500K Special Roadster	Vintage Cars	1:18	Studio M Art Models	8635	24.26	53.91

10. Create a function to get the creditLimit of customers less than 96800.

What the Function Does:

1. **Takes a parameter** (e.g., @Limit) which specifies the upper bound for credit limits.
2. **Queries the customers table** to return customers whose creditLimit is **less than** this value.
3. Returns a **table** containing customer details (such as customerNumber, customerName, and creditLimit).

SQLQuery3.sql - DE...NEAPVJ\Henit (57))* | TaskQueries.sql -...DNEAPVJ\Henit (52)) | SQLQuery1.sql - DE.

```sql
CREATE FUNCTION fn_GetCustomersBelowCreditLimit
(
    @limit FLOAT  -- Input parameter to define the credit limit threshold
)
RETURNS TABLE
AS
RETURN
(
    SELECT
        customerNumber,
        customerName,
        creditLimit
    FROM customers
    WHERE creditLimit < @limit
);

-- Get customers with credit limit less than 96800
SELECT * FROM dbo.fn_GetCustomersBelowCreditLimit(96800);
```

100 %

Results | Messages

	customerNumber	customerName	creditLimit
1	103	Atelier graphique	21000
2	112	Signal Gift Stores	71800
3	121	Baane Mini Imports	81700
4	125	Havel & Zbyszek Co	0
5	128	Blauer See Auto, Co.	59700
6	129	Mini Wheels Co.	64600
7	144	Volvo Model Replicas, Co	53100
8	145	Danish Wholesale Imports	83400
9	161	Technics Stores Inc.	84600

- This is an inline table-valued function, which performs better than multi-statement functions.
- Returns structured output to be joined or filtered further if needed
- We can easily change the parameter to get customers below any credit limit.

11. Create Trigger to store transaction record for employee table which displays employeeNumber, lastName, FirstName and office code upon insertion

How It Works:

- AFTER INSERT: Trigger activates only after an insert is successfully done.
- inserted: This is a special SQL Server table that temporarily holds the newly inserted rows during the trigger execution.
- It logs each inserted employee's details into employee_insert_log.

Step 1: Create a Log Table
First, you need a table to store the inserted employee transaction records:

SQLQuery4.sql - DE...NEAPVJ\Henit (63))* | SQLQuery3.sql - DE...NEAPVJ\Henit (57))*

```sql
-- Create a log table to store inserted employee records
CREATE TABLE employee_insert_log (
    logId INT IDENTITY(1,1) PRIMARY KEY,
    employeeNumber INT,
    lastName VARCHAR(50),
    firstName VARCHAR(50),
    officeCode VARCHAR(10),
    insertDateTime DATETIME DEFAULT GETDATE()
);
```

100 %

Messages

```
Commands completed successfully.

Completion time: 2025-05-15T08:56:53.6817500-05:00
```

Step 2: Create the Trigger

SQLQuery4.sql - DE...NEAPVJ\Henit (63))* | SQLQuery3.sql - DE...NEAPVJ\Henit (57))* | TaskQueries.sql -...DNEA

```sql
-- Create Trigger that fires after an insert on the 'employees' table
CREATE TRIGGER trg_AfterInsert_Employee
ON employees
AFTER INSERT
AS
BEGIN
    -- Insert new employee records into the log table
    INSERT INTO employee_insert_log (employeeNumber, lastName, firstName, officeCode)
    SELECT
        employeeNumber,
        lastName,
        firstName,
        officeCode
    FROM inserted;

    PRINT 'Employee insert transaction recorded.';
END
```

100 %

Messages

```
Commands completed successfully.

Completion time: 2025-05-15T08:58:26.1364499-05:00
```

Step3: Test the Trigger

SQLQuery4.sql - DE...NEAPVJ\Henit (63))* | SQLQuery3.sql - DE...NEAPVJ\Henit (57))* | TaskQueries.sql -...DNEAPVJ\Henit (52)) | SQLQue

```sql
-- Insert a sample employee to trigger the insert log
INSERT INTO employees (employeeNumber, lastName, firstName, extension, email, officeCode, reportsTo, jobTitle)
VALUES (2001, 'Smith', 'John', 'x1234', 'john.smith@example.com', '1', NULL, 'Sales Rep');

-- Check the log
SELECT * FROM employee_insert_log;
```

100 %

Results | Messages

	logId	employeeNumber	lastName	firstName	officeCode	insertDateTime
1	1	2001	Smith	John	1	2025-05-15 09:00:25.120

12. Create a Trigger to display customer number if the amount is greater than 10,000.

To create a trigger that responds when a payment amount exceeds 10,000, we can define an AFTER INSERT trigger on the payments table. This will check inserted values and perform an action — in this case, display the customerNumber.

How It Works:

- inserted is a virtual table containing rows just inserted into payments.
- It checks whether any row has amount > 10000.
- If such rows exist, it selects and displays customerNumber and amount.

SQLQuery5.sql - DE...NEAPVJ\Henit (66))* | TaskQueries.sql -...DNEAPVJ\Henit (52)) | SQLQuery1.sql - DE...NEAPVJ\He

```sql
-- Trigger to display customerNumber if amount > 10,000
CREATE TRIGGER trg_CheckHighPayment
ON payments
AFTER INSERT
AS
BEGIN
    -- Check if any inserted payment has amount > 10,000
    IF EXISTS (
        SELECT 1
        FROM inserted
        WHERE amount > 10000
    )
    BEGIN
        -- Print customer numbers with high payment amount
        SELECT customerNumber, amount
        FROM inserted
        WHERE amount > 10000;

        PRINT 'Payment over 10,000 detected.';
    END
END;

-- Inserting a payment with amount > 10,000
INSERT INTO payments (customerNumber, checkNumber, paymentDate, amount)
VALUES (103, 'CHK10001', GETDATE(), 15000);
```

100 %

Results | Messages

	customerNumber	amount
1	103	15000

13. Create Users, Roles and Logins according to 3 Roles: Admin, HR, and Employee. Admin can view full database and has full access, HR can view and access only employee and offices table. Employee can view all tables only.

Role	Permissions
Admin	Full DB access (CONTROL)
HR	Read/write on employees, offices
Employee	Read-only access to all tables

To create users, roles, and logins with the specified access levels in SQL Server, we need to follow these steps in order. This ensures correct assignment of permissions based on roles:

Step 1: Create Logins
These are SQL Server-level accounts.

SQLQuery6.sql - DE...NEAPVJ\Henit (62))* | TaskQueries.sql -...DNEAPVJ\Henit (52))

```sql
-- Create Logins for Admin, HR, and Employee
CREATE LOGIN login_admin WITH PASSWORD = 'Admin@123';
CREATE LOGIN login_hr WITH PASSWORD = 'HR@123';
CREATE LOGIN login_employee WITH PASSWORD = 'Employee@123';
```

100 %

Messages

```
Commands completed successfully.

Completion time: 2025-05-15T10:12:34.4170853-05:00
```

Step 2: Create Database Users
These map the logins to the current database.

```sql
--  Create Database Users
USE MotorsCertification

CREATE USER user_admin FOR LOGIN login_admin
CREATE USER user_hr FOR LOGIN login_hr
CREATE USER user_employee FOR LOGIN login_employee
```

100 %

Messages

```
Commands completed successfully.

Completion time: 2025-05-15T10:14:44.3813266-05:00
```

Step 3: Create Database Roles

```sql
-- Create roles
CREATE ROLE role_admin;
CREATE ROLE role_hr;
CREATE ROLE role_employee;
```

100 %

Messages

```
Commands completed successfully.

Completion time: 2025-05-15T10:16:19.8721258-05:00
```

Step 4: Assign Users to Roles

SQLQuery6.sql - DE...NEAPVJ\Henit (62))* | TaskQueries.sql -...DNEAPVJ\Henit

```sql
--  Add users to respective roles
ALTER ROLE role_admin ADD MEMBER user_admin;
ALTER ROLE role_hr ADD MEMBER user_hr;
ALTER ROLE role_employee ADD MEMBER user_employee;
```

100 %

Messages

```
Commands completed successfully.

Completion time: 2025-05-15T10:18:51.8997167-05:00
```

Step 5: Grant Permissions

Admin Role: Full Access

HR Role: Read and Write on employees and offices only

Employee Role: Read-Only on All Tables

```sql
USE MotorsCertification
-- Grant full access to admin role
GRANT CONTROL ON DATABASE::MotorsCertification TO role_admin;


-- Grant SELECT, INSERT, UPDATE, DELETE on employees and offices
GRANT SELECT, INSERT, UPDATE, DELETE ON dbo.employees TO role_hr;
GRANT SELECT, INSERT, UPDATE, DELETE ON dbo.offices TO role_hr;

-- Grant read-only access to all tables
-- This dynamically generates GRANT SELECT for each table in the DB
DECLARE @sql NVARCHAR(MAX) = N'';
SELECT @sql += 'GRANT SELECT ON [' + s.name + '].[' + t.name + '] TO role_employee;'
FROM sys.tables t
JOIN sys.schemas s ON t.schema_id = s.schema_id;

EXEC sp_executesql @sql;
```

100 %

Messages

```
Commands completed successfully.

Completion time: 2025-05-15T10:23:42.7096334-05:00
```

Check All Database Users and Roles

```sql
--1. Check All Database Users and Roles List all users and their roles
SELECT
    r.name AS RoleName,
    m.name AS UserName
FROM
    sys.database_role_members rm
JOIN
    sys.database_principals r ON rm.role_principal_id = r.principal_id
JOIN
    sys.database_principals m ON rm.member_principal_id = m.principal_id
ORDER BY RoleName
```

100 %

Results | Messages

	RoleName	UserName
1	db_owner	dbo
2	role_admin	user_admin
3	role_employee	user_employee
4	role_hr	user_hr

Verify Login to User Mapping -- Shows database users and the SQL Server logins they map to

```sql
-- Verify Login to User Mapping -- Shows database users and the SQL Server logins they map to
SELECT
    dp.name AS DatabaseUser,
    dp.type_desc AS UserType,
    sp.name AS LoginName
FROM
    sys.database_principals dp
LEFT JOIN
    sys.server_principals sp ON dp.sid = sp.sid
WHERE
    dp.type IN ('S', 'U') AND dp.authentication_type = 1
```

100 %

Results | Messages

	DatabaseUser	UserType	LoginName
1	user_admin	SQL_USER	login_admin
2	user_hr	SQL_USER	login_hr
3	user_employee	SQL_USER	login_employee

Test Permissions for Specific Role/User

To test what a specific user (e.g. user_hr) can do, use the EXECUTE AS clause:

```sql
--Test Permissions for Specific Role/User
--To test what a specific user (e.g. user_hr) can do, use the EXECUTE AS clause:

-- Switch to HR user context
EXECUTE AS USER = 'user_hr';

-- Try selecting from employees (should work)
SELECT TOP 1 * FROM employees;

-- Try selecting from orders (should fail for HR)
SELECT TOP 1 * FROM orders;

-- Revert context
REVERT;
```

100 %

Results | Messages

```
(1 row affected)
Msg 229, Level 14, State 5, Line 98
The SELECT permission was denied on the object 'orders', database 'MotorsCertification', schema 'dbo'.

Completion time: 2025-05-15T10:37:53.2034765-05:00
```